Stroke Data Analysis

Esmma Almousa

Stroke Data Analysis with General Linear Model and Random Forest

Data Cleaning and Preparation for GLM Model:

Load Data:

```
stroke_data <- read_csv("stroke_data.csv")
```

```
## Rows: 43400 Columns: 15
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr  (5): gender, married, occupation, residence, smoking_status
## dbl (10): id, age, hypertension, heart_disease, metric_1, metric_2, metric_3...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Checking for NA values in each variable:

```
colSums(is.na(stroke_data)) # It is noted that there are missing variables in metric_
2 and smoking_status
```

```
##             id         gender            age        married   hypertension
##              0              0              0              0              0
##  heart_disease     occupation      residence       metric_1       metric_2
##              0              0              0              0           1462
##       metric_3       metric_4       metric_5 smoking_status         stroke
##              0              0              0          13292              0
```

Finding Unique Values

```
 # Determining unique possible values of variables for the next step
unique(stroke_data$gender)
```

```
## [1] "Male"   "Female" "Other"
```

```
unique(stroke_data$married)
```

```
## [1] "No"  "Yes"
```

```
unique(stroke_data$occupation)
```

```
## [1] "A" "B" "C" "D" "E"
```

```
unique(stroke_data$residence)
```

```
## [1] "Rural" "Urban"
```

```
unique(stroke_data$smoking_status)
```

```
## [1] NA                "never smoked"    "formerly smoked" "smokes"
```

Convert Categorical Data to Numeric Values for GLM Model:

```
edited_stroke_data <- stroke_data %>% #tidyverse library used to mutate current data

  mutate(gender_numeric = case_when(gender == "Male" ~ 0,
                                    gender == "Female" ~ 1,
                                    TRUE ~ 2),
         married_numeric = case_when(married == "No"~0,
                                     married == "Yes"~1),
         occupation_numeric =case_when(occupation =="A"~0,
                                       occupation =="B"~1,
                                       occupation =="C"~2,
                                       occupation =="D"~3,
                                       occupation == "E"~4
                                         ), # Converting categorical data to numeric v
alues
         residence_numeric = case_when(residence=="Rural"~0,
                                       residence=="Urban"~1
                                       ),
         smoking_numeric = case_when(
           smoking_status =="never smoked"~0,
           smoking_status == "formerly smoked"~1,
           smoking_status == "smokes"~2,
           TRUE ~ 3) # All NA values in smoking status are replaced with the number
3, which represents the value for unknowns.
         ) %>%
  select(-id) # id number is removed from set in order to avoid its interference with
the predictive model
```

Metric_2 Missing values are Imputed:

```
# categorical data is dropped in order to perform na.roughfix for missing values in m
etric_2
numeric_stroke_data <- edited_stroke_data[,sapply(edited_stroke_data, is.numeric)]

numeric_stroke_data$metric_2 <- na.roughfix(numeric_stroke_data$metric_2) # Due to la
ck of information about metric_2, NA's will be replaced by the median value of the va
riable in order to avoid dropping other useful data for the predictive model
```

```
colSums(is.na(numeric_stroke_data)) # Numeric cleaned dataset now has no NA values an
d is ready to be used for the GLM Model
```

```
##                 age        hypertension       heart_disease             metric_1
##                   0                   0                   0                   0
##            metric_2             metric_3             metric_4             metric_5
##                   0                   0                   0                   0
##              stroke      gender_numeric   married_numeric occupation_numeric
##                   0                   0                   0                   0
##  residence_numeric     smoking_numeric
##                   0                   0
```

Data Cleaning and Preparation Summary:

First, missing values within each variable were detected for the purpose of substitution. I chose to substitute missing values instead of dropping the entire row in order to utilize as much observational data for the predictive model as possible. All categorical data in the original set were replaced with numerical values. For example, in the variable “Gender”, 0 represents “Males”, 1 represents “Females”, and 2 represents “Other”. Variables were mannually mutated using the tidyverse package in order to replace all possible categorical variables with an assigned numerical value. One-hot-encoding was not used in order to avoid an increased number of variables and multicollinearity between them. It was noted that the smoking_status had 13292 missing variables and metric_2 had approximately 1462 missing values. Smoking_status was replaced with the numeric value “3” to represent a fourth category as “Unknown”. Missing values in Metric_2 were replaced with the median value of the variable’s column using na.roughfix since information about this metric is not provided and to avoid row dropping. Additionally, the variable “id” was dropped in order to avoid input of this variable in the predictive model. After these mutations, the data is now ready to be used for a GLM model.

GLM Model:

Preparation for splitting training and test set

```
levels(numeric_stroke_data$stroke) <- 0:1
set.seed(123) #Setting seed allows for a fixed starting point
split_data <- sample.split(numeric_stroke_data$stroke, SplitRatio = 0.80)
```

```
trainingSet <- subset(numeric_stroke_data, split_data == TRUE) # Training set is 80% of data
testSet <- subset(numeric_stroke_data, split_data == FALSE) # Test set is remaining 20%
```

GLM Model and Summary:

```
Stroke_glm <- glm(stroke ~ gender_numeric + age + married_numeric + hypertension + heart_disease + occupation_numeric + residence_numeric + metric_1 + metric_2 + metric_3 + metric_4  + smoking_numeric, data = trainingSet, family = binomial())
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
# metric_5 was not incorporated in the model because it is proportional to metric_1
summary(Stroke_glm)
```

```
##
## Call:
## glm(formula = stroke ~ gender_numeric + age + married_numeric +
##     hypertension + heart_disease + occupation_numeric + residence_numeric +
##     metric_1 + metric_2 + metric_3 + metric_4 + smoking_numeric,
##     family = binomial(), data = trainingSet)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -8.4904  -0.2022  -0.1221  -0.0707   3.8033
##
## Coefficients:
##                      Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -3.4717411  2.7594620  -1.258 0.208347
## gender_numeric     -0.0246286  0.0843909  -0.292 0.770409
## age                 0.0534727  0.0029677  18.018  < 2e-16 ***
## married_numeric     0.0688367  0.1400327   0.492 0.623019
## hypertension        0.4403736  0.0979930   4.494 6.99e-06 ***
## heart_disease       0.7050612  0.1078065   6.540 6.15e-11 ***
## occupation_numeric  0.0045776  0.0350047   0.131 0.895956
## residence_numeric   0.0345955  0.0824217   0.420 0.674677
## metric_1            0.0039453  0.0007454   5.293 1.20e-07 ***
## metric_2           -0.0256272  0.0068978  -3.715 0.000203 ***
## metric_3           -0.0488169  0.0920059  -0.531 0.595707
## metric_4           -0.0343720  0.0281444  -1.221 0.221983
## smoking_numeric    -0.0268651  0.0370643  -0.725 0.468560
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 6268.3  on 34719  degrees of freedom
## Residual deviance: 5319.5  on 34707  degrees of freedom
## AIC: 5345.5
##
## Number of Fisher Scoring iterations: 9
```

GLM Model Applied to Test Set:

```
Prediction_Model <- predict(Stroke_glm, testSet, type = "response")
```

Using Values from Prediction Model to Assign Participants to 0 or 1

```
predictions <- data.frame(Prediction_Model) # Outcomes from model placed in dataframe

predictions <- predictions %>%
  mutate(stroke = case_when(predictions[,1] > .025 ~ 1,
                            TRUE ~0)) # If value is above the cutoff in the predictio
n dataframe is above a certain cutoff, participant is assigned to 1 for stroke, other
wise, participant is assigned to 0
head(predictions)
```

```
##   Prediction_Model stroke
## 1      0.025641275      1
## 2      0.002559619      0
## 3      0.134372038      1
## 4      0.042136900      1
## 5      0.003934058      0
## 6      0.095366260      1
```

Confusion Matrix for GLM Model's Performance:

```
confusionMatrix(as.factor(predictions$stroke),as.factor(testSet$stroke))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 6766   44
##          1 1757  113
##
##                Accuracy : 0.7925
##                  95% CI : (0.7838, 0.801)
##     No Information Rate : 0.9819
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.0808
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.79385
##             Specificity : 0.71975
##          Pos Pred Value : 0.99354
##          Neg Pred Value : 0.06043
##              Prevalence : 0.98191
##          Detection Rate : 0.77949
##    Detection Prevalence : 0.78456
##       Balanced Accuracy : 0.75680
##
##        'Positive' Class : 0
##
```

GLM Model Results:

Based on the confusion matrix shown above, this model has an accuracy rate of 79.25%. The response variable was first split into two levels: 0 and 1. The split ratio for the training and test set was then assigned, where 80% of data would be assigned to the training set, and 20% would be assigned to the test set. A glm model was then designed using the training set, where stroke is predicted using all other variables in the dataset. Metric_5 was disregarded in the model because it is proportional to Metric_1. The GLM model was then used to predict results for the test set. Predicted outcomes were placed in a dataframe and rounded based on the assigned cutoff of .025.

GLM Future Approaches:

With more time, I would explore another GLM model using the significant model coefficients. The coefficient table for the GLM model showed that age, hypertension, heart disease, metric_1, and metric_2 all had significant positive influence on stroke. I would be interested in using these significant variables and comparing the accuracy with the current one that includes all other variables. I would also search for a more optimal cut off from the prediciton model that would best classify the two levels of stroke on the test set.

Alternate Model: Random Forest:

```
# Since RF can handle categorical variables, original dataset variables will be used
rf_data <- stroke_data %>%
  mutate(smoking_numeric = case_when(
            smoking_status =="never smoked"~0,
            smoking_status == "formerly smoked"~1,
            smoking_status == "smokes"~2,
            TRUE ~ 3)) %>%  # Accounts for missing values in smoking_status
  select(-smoking_status,-id) # smoking_status variable now dropped after updated var
iable has been created.
rf_data$metric_2 <- na.roughfix(rf_data$metric_2) # missing metric_2 values are repla
ced with median value of column
```

```
data = data.frame(rf_data, StringsAsFactors = TRUE) # Ensures Strings are converted t
o factors for RF preparation
```

```
factor.columns <- sapply(data,function(x)
  class(x) != 'integer' &
  class(x) != 'numeric') # Takes input and converts it to a matrix for purpose of RF
use

data[,factor.columns] <- data.frame(apply(data[,factor.columns,drop=F], 2, as.facto
r),stringsAsFactors = TRUE)
```

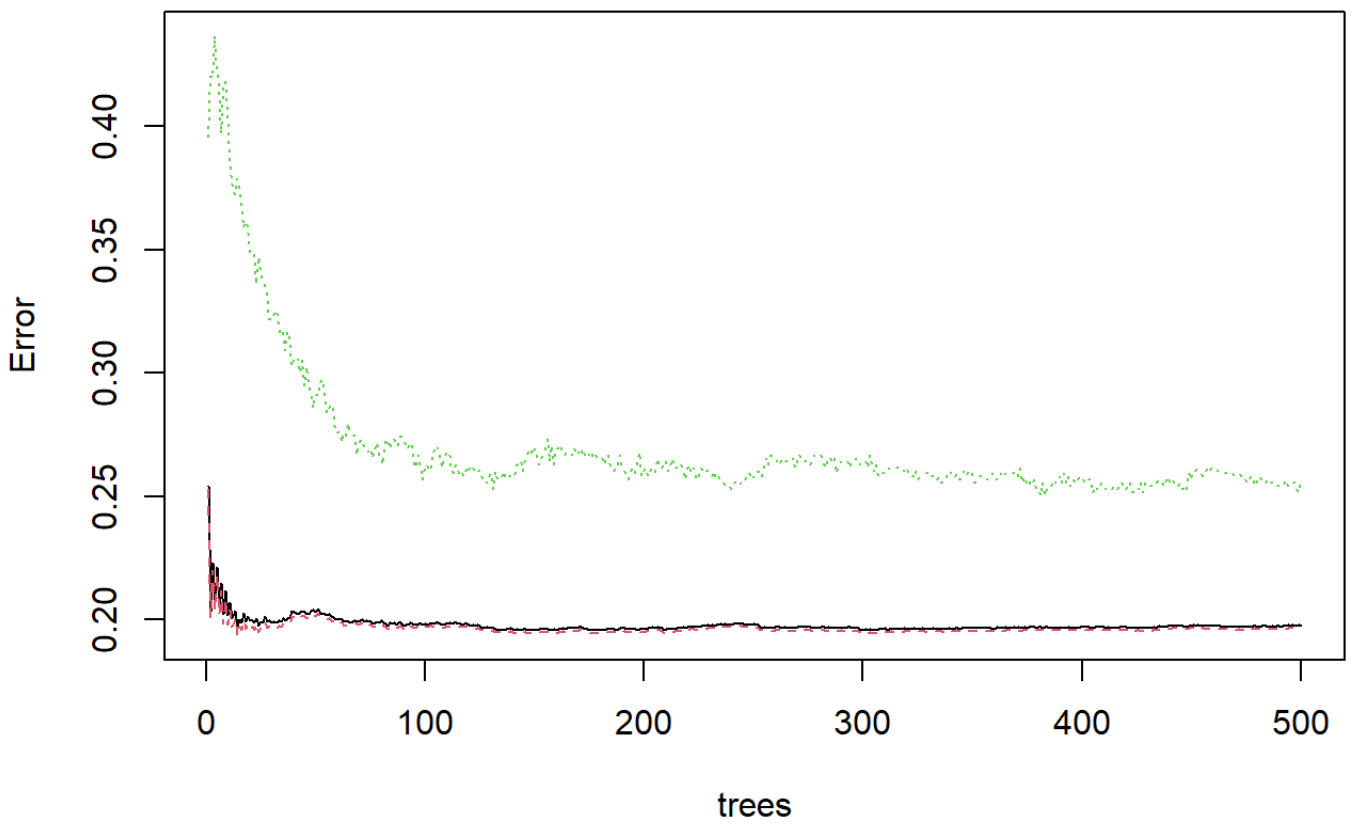
```
response <- "stroke"

# Assigns response variables and predictor variables that will be used in RF Model an
d ensures there will be no missing values in the response variable
response_var <- data[!is.na(data[response]), colnames(data) == response]
predictor_vars <- data[!is.na(data[response]), colnames(data) != response]

# Impute missing values in predictor variables
predictor_vars <- na.roughfix(predictor_vars)
set.seed(123)
response_var = as.factor(response_var) # convert response variable to factor
min_size = min(table(response_var))
num_classes = length(table(response_var)) # down sampling will be used to avoid
```

```
RF_Model = rfExtra::randomForest(x = predictor_vars, y = response_var,
    importance = TRUE, ntree = 500,
    sampsize = rep(min_size, num_classes))
```

```
RF_Model
```

```
##
## Call:
##  randomForest(x = predictor_vars, y = response_var, ntree = 500,      sampsize = r
ep(min_size, num_classes), importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 19.77%
## Confusion matrix:
##       0    1 class.error
## 0 34238 8379   0.1966117
## 1   201  582   0.2567050
```

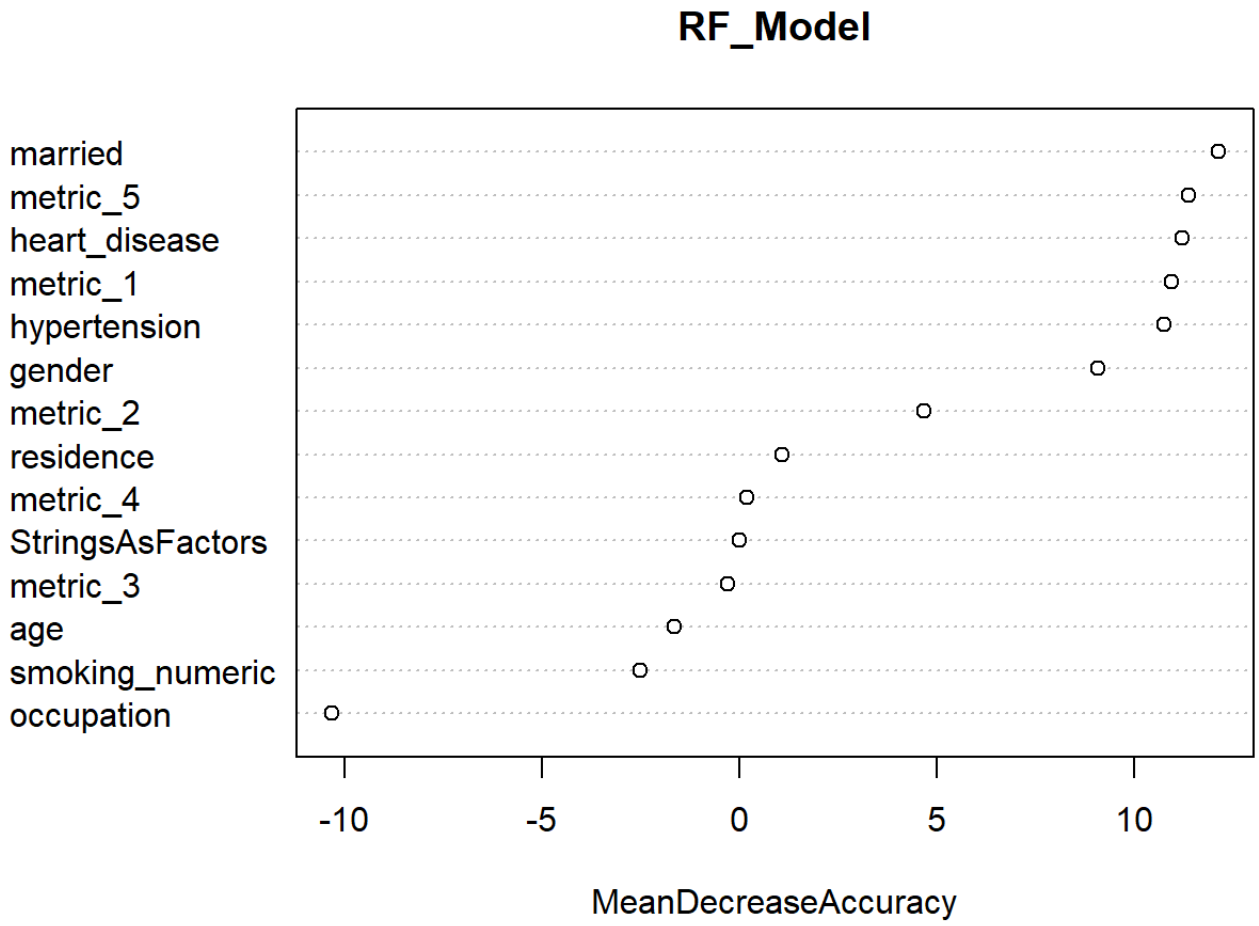
```
plot(RF_Model) # plot to demonstrate how error changes as number of tree of forest in
creases
```




```
orderofimp<- data.frame(importance(RF_Model))
Stroke_Order_of_Imp <- orderofimp[order(-orderofimp$MeanDecreaseAccuracy),]
Stroke_Order_of_Imp # Sort variables in order of importance for RF Model.
```

```
##                           X0         X1 MeanDecreaseAccuracy MeanDecreaseGini
## married           12.0301264  3.5320042           12.1309513         28.33518
## metric_5          11.1345752 13.7829044           11.3756949         83.76304
## heart_disease     10.7995736 17.7786887           11.2323681         23.15882
## metric_1          10.6834621 14.4338472           10.9534197         83.99784
## hypertension      10.6624912  5.0503152           10.7701546         17.75393
## gender             8.9377362  2.3255995            9.0800640         12.01094
## metric_2           4.3167930 16.1773993            4.6916376         87.17880
## residence          1.1328644 -0.4926903            1.0840529         12.19073
## metric_4           0.2105119 -0.1370516            0.1937474         74.44576
## StringsAsFactors   0.0000000  0.0000000            0.0000000          0.00000
## metric_3          -0.4257173  1.3439017           -0.2952088         12.24173
## age               -3.4022639 87.8288298           -1.6562075        238.38574
## smoking_numeric   -2.8188437  7.9684976           -2.5063276         28.07713
## occupation       -10.4333855 14.5301084          -10.3098056         38.69914
```

```
varImpPlot(RF_Model,,type=1)
```



Alternatve Model Summary, Results, and Future Approaches:

The accuracy of this model is approximately 80.3%. Random Forest was selected as the alternate model because it is known to be a strong classifier in research for predictive models regarding health risks and diseases. RF also comes with the advantage of having a lower risk for overfitting compared to GLM models. Missing values were handled the same way that NA values were handled within the GLM model. Since RF can handle both numeric and categorical variables, the original variables from the provided dataset were used in this model.

A disadvantage of Random Forest is that it leads to slower performance due to the high number of trees, which is why I only used 500 trees total for this model. In the future, I would perform random forest using parallel methods with more trees to improve both accuracy and run-time. I am also interested in using feature selection to incorporate the most important variables for prediction and determine the appropriate variables that should be used for the lowest possible error rate.